COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	09/15/26	**DATE SUBMITTED:**	09/15/26
NAME:	Cruz, Elijah J	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 8.1 Sorting Algorithms Pt.2

1. PROCEDURE OUTPUT

Preparation Task:

- Create an array of elements with random values. The array must contain 100 elements that are not sorted. Use the created array for each sorting algorithm below. Show the output of the preparation task on table 8-1 in section 6.
- Create a header file for the implementation of the different sorting algorithms.
- Import this header file into your main.cpp.

Code + Console Screenshot	**main.cpp** CMakeLists.txt × main.cpp × sorting.h

```cpp
//Cruz Elijah
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"
using namespace std;

int main() {
    const int SIZE = 100;
    int original[SIZE];

    // generate 100 random, unsorted values
    srand( Seed: time(0));
    for (int i = 0; i < SIZE; i++) {
        original[i] = rand() % 1000;
    }

    cout << "Preparation Task: Original Unsorted Array" << endl;
    printArray(original, SIZE);

    // Shell Sort
```

	Sorting.h
	`// Prints all elements of the array, 10 per line`
	`void printArray(int arr[], int n) {`
	`  for (int i = 0; i < n; i++) {`
	`  cout << arr[i] << " ";`
	`  if ((i + 1) % 10 == 0) cout << endl;`
	`  }`
	`  cout << endl;`
	`}`
	OUTPUT
	Run SORT_SUPP2
	"C:\Users\Elijah\CLionProjects\SORT SUPP2\cmake-build-debug\SORT_SUPP2.exe"
	Preparation Task: Original Unsorted Array
	904 685 185 410 106 895 184 312 725 129
	208 226 799 26 699 195 668 216 975 715
	72 424 881 685 372 524 990 385 971 60
	16 405 748 280 806 608 826 997 677 679
	547 382 998 707 518 365 8 58 393 114
	232 606 754 987 493 19 468 733 265 732
	828 778 741 217 812 3 212 626 719 167
	140 607 126 277 968 106 579 159 601 125
	805 528 589 161 323 335 872 230 388 708
	755 754 6 178 779 84 790 581 572 992

Observations	Contains the output and the code for generating 100 random elements for the array that will be used by the ff. sorting algorithm and techniques. As observed, we can see the created unsorted random elements with the highest being 998.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	**main.cpp**
	`// Shell Sort`
	`int shellArr[SIZE];`
	`for (int i = 0; i < SIZE; i++) {`
	`  shellArr[i] = original[i];`
	`}`
	`shellSort(shellArr, SIZE);`
	`cout << "\nShell Sort" << endl;`
	`printArray(shellArr, SIZE);`
	Sorting.h

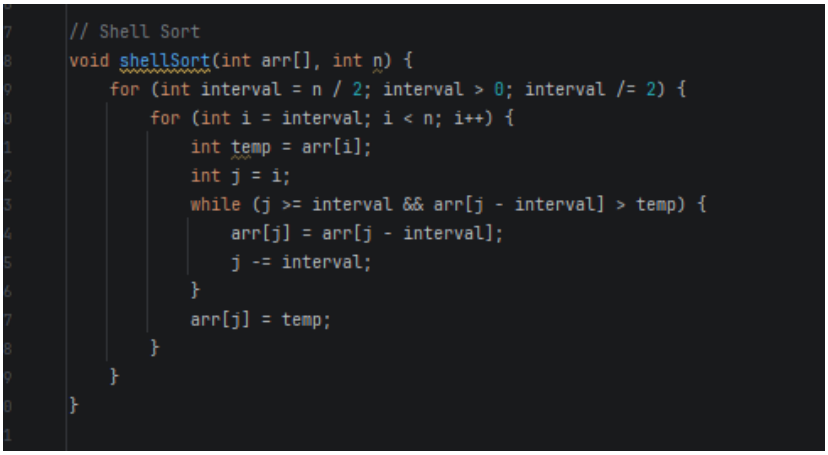
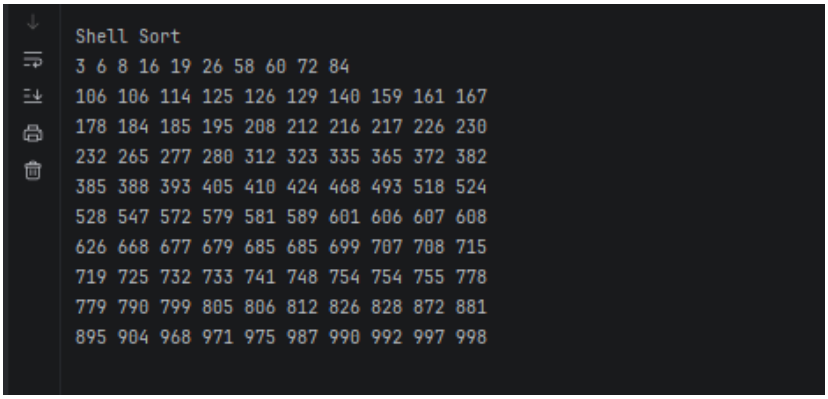
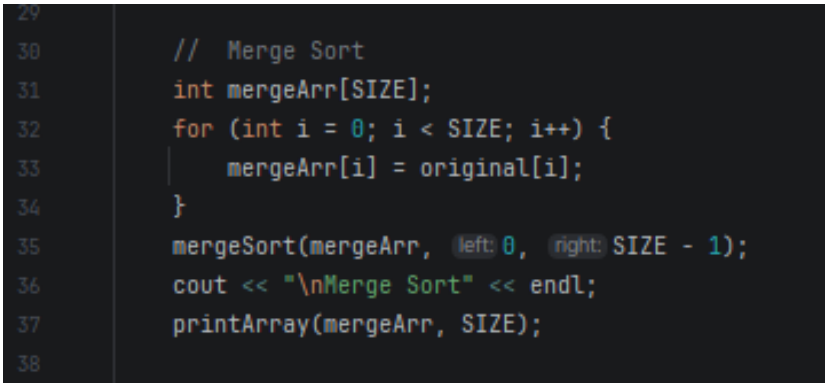
	 **OUTPUT** 
Observations	Contains snippets of the code for the shell sorting technique wherein the process of sorting the elements are accompanied by a gap and then runs an insertion sort from this said gap in each interval and iteration.

Table 8-2. Shell Sort Technique

Code + Console Screenshot	**main.cpp**  **Sorting.h**

```cpp
// Merge Sort
void merge(int arr[], int left, int middle, int right) {
    int n1 = middle - left + 1;
    int n2 = right - middle;
    vector<int> leftArr(n1), rightArr(n2);

    for (int i = 0; i < n1; i++) leftArr[i] = arr[left + i];
    for (int j = 0; j < n2; j++) rightArr[j] = arr[middle + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (leftArr[i] <= rightArr[j]) arr[k++] = leftArr[i++];
        else arr[k++] = rightArr[j++];
    }
    while (i < n1) arr[k++] = leftArr[i++];
    while (j < n2) arr[k++] = rightArr[j++];
}
```

OUTPUT

```
Merge Sort
3 6 8 16 19 26 58 60 72 84
106 106 114 125 126 129 140 159 161 167
178 184 185 195 208 212 216 217 226 230
232 265 277 280 312 323 335 365 372 382
385 388 393 405 410 424 468 493 518 524
528 547 572 579 581 589 601 606 607 608
626 668 677 679 685 685 699 707 708 715
719 725 732 733 741 748 754 754 755 778
779 790 799 805 806 812 826 828 872 881
895 904 968 971 975 987 990 992 997 998
```

Observations	Contains snippets of the code for the merge sort algorithm that follows the process of breaking down the array into sub arrays, sorts and compares the elements, and then reinserts the elements back into one single sorted output array. The same output is achieved after the runtime when compared to the output of the Shell sort algorithm or sorting technique.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot

main.cpp

```
    //  Quick Sort
    int quickArr[SIZE];
    for (int i = 0; i < SIZE; i++) {
        quickArr[i] = original[i];
    }
    quickSort(quickArr, low: 0, high: SIZE - 1);
    cout << "\nQuick Sort" << endl;
    printArray(quickArr, SIZE);

    return 0;
}
```

Sorting.h

```
// Quick Sort
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap( a: &arr[i], b: &arr[j]);
        }
    }
    swap( a: &arr[i + 1], b: &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivot = partition(arr, low, high);
        quickSort(arr, low, high: pivot - 1);
        quickSort(arr, low: pivot + 1, high);
    }
}

#endif  #ifndef SORTING_H
```

OUTPUT

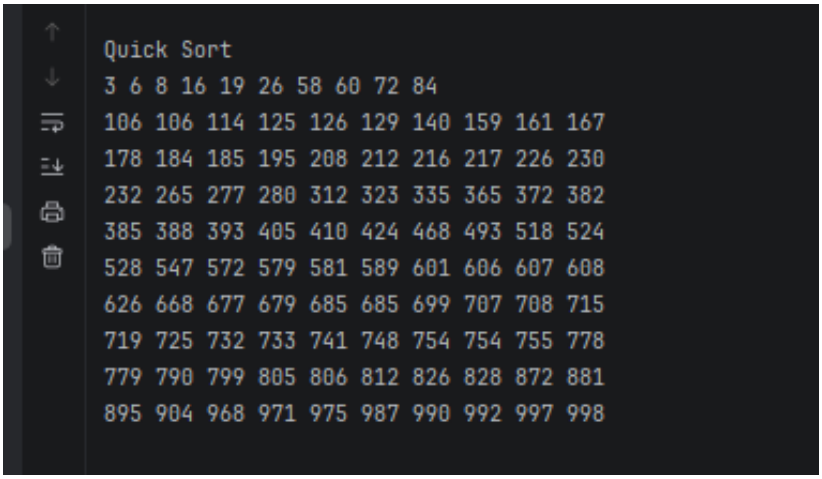
	
Observations	Contains the snippets for the code and output of the quick sort algorithm. As observed, we achieve the same output when compared to different sorting algorithms such as the merge and shell sort. The quick sort algorithm follows the process of picking a pivot to act as a reference, partitioning by rearranging the array, sorting by comparing the elements in each interaction and then stops the recursion when a sub array has zero or one element remaining. Meaning the array is sorted.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Yes. As once partition() runs, the left side is all ≤ pivot, the right side is all ≥ pivot, and the pivot is already in its final spot. Those two sides are now independent arrays, so any sorting algorithm can sort them, not just quicksort To demonstrate this, the left and right sub-lists were sorted using Shell Sort instead of Quick Sort:

```
//Problem 1: Quick Sort using Shell Sort on the sub-lists
void TestQuickSort(int arr[], int low, int high) {
   if (low < high) {
       int pivot = partition(arr, low, high);
       shellSort(arr + low, pivot - low);        // sort the left sub-list
       shellSort(arr + pivot + 1, high - pivot);  // sort the right sub-list
   }
}
```

Using the array {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}, this version produced a correctly sorted result:

```
"C:\Users\Elijah\CLionProjects\TEST SORT\cmake-build-debug\TEST_SORT.exe"
Original array:
4 34 29 48 53 87 12 30 44 25
93 67 43 19 74

Sorted array (Quick Sort partition + Shell Sort on sub-lists):
4 12 19 25 29 30 34 43 44 48
53 67 74 87 93

Process finished with exit code 0
```

Which confirms that the sub-lists produced by partition() can be sorted by a different algorithm, sorting each side independently still results in the full array being sorted correctly.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

Shell Sort is the fastest for this array as it doesn't require extra memory such as merge and quick sort does in their process of sorting that uses temp and sub arrays to store these elements before sorting them. However, this doesn't mean that the mentioned sorting algorithms are slow in every type and volume of data entered. And Merge sort and quicksort hit O(N log N) because each cuts the array roughly in half every recursive step. That's log N levels, and each level does O(N) total work (merging or partitioning). N × log N = O(N log N). Merge sort's split is always exact, so this holds every time with the Quicksort's split depending on the pivot.

CONCLUSION

In conclusion, this hands-on-activity focuses on how each algorithm technique (shell, quicksort, and merge) works and how they can be implemented in different scenarios. In addition, it also allows us to learn more of their advantages and disadvantages with shell sort being the fastest for the array of elements for the supplementary task. Focusing more of their strengths and weaknesses when it comes to sorting large amounts of data. Overall, the three sorting algorithms for this activity were able to produce the same output in sorting the randomly generated array of elements, with the only difference present is the time it takes for them to sort the elements in each iteration.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: **Grammar correction, debugging**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Gemini was used for code debugging and grammar correction in producing observations and conclusions for this activity.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.